# GovBiz

## AI 서비스 기술 보고서

정부지원사업 탐색·신청 관리 플랫폼

### 에이전트 실행부터
### 품질 평가와 배포·운영까지

# 목차

프로젝트 개요부터 아키텍처와 구현 상세까지


01 **프로젝트 개요** 03
서비스 소개 · 사용자와 관리자 흐름

02 **시스템 아키텍처** 05
접속 · 서비스 · 평가 · 배포 구성

03 **LLM 기반 라우팅** 12
공고 선택 후 요청과 업무 연결

04 **LLMOps** 14
평가 · 추적 · 실행 관리

05 **EKS · GitOps** 20
Helm · Argo CD 배포 자동화

06 **마이크로서비스** 22
Core · Catalog · AI · Ops

07 **웹·모바일 공유 구조** 23
React · React Native

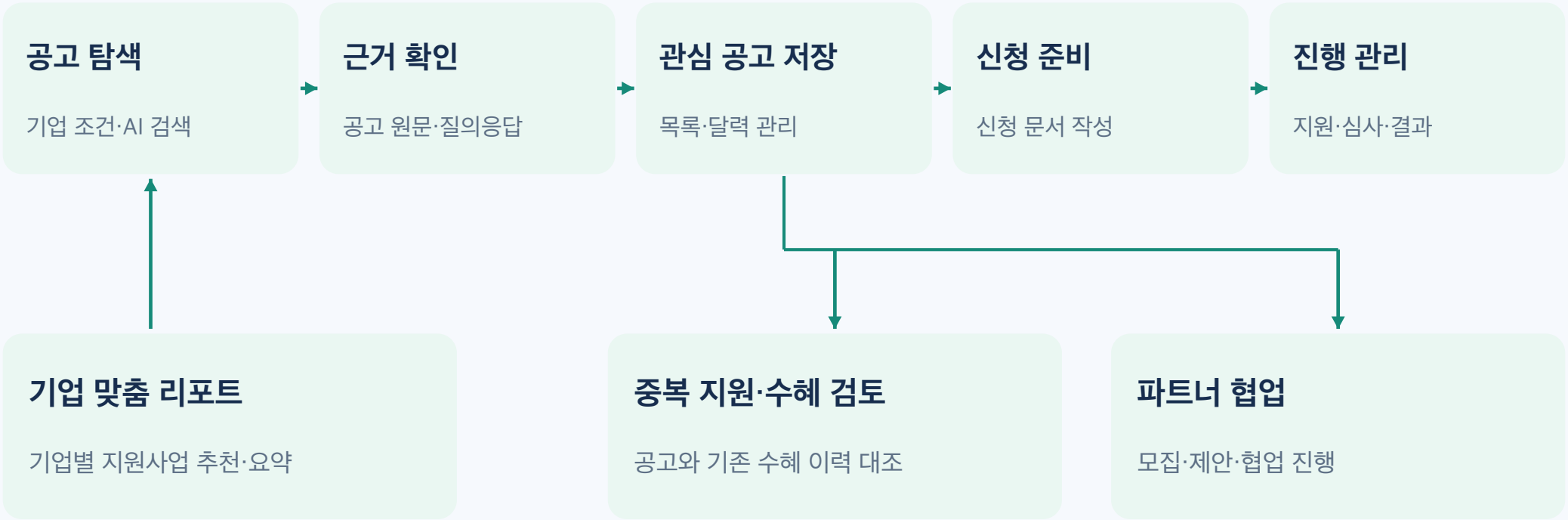
08 **부록** 25
README 전체 시스템 아키텍처

# 공고 탐색부터 신청 준비까지, GovBiz

기업의 조건에 맞는 정부지원사업을 찾고, 공식 근거 확인부터 신청 준비·진행 관리까지 연결하는 AI 업무 플랫폼입니다.

3차의 비즈니스 기능을 계승하고, 4차에서는 모바일 이용 채널과 GitOps 배포 체계를 추가했습니다.

## 사용자 · 지원사업을 찾고 신청을 준비합니다



# 관리자 사용 흐름

실제 서비스 운영과 테스트 평가를 구분해 기록을 확인하고 품질을 관리합니다.

## 운영 · 실제 서비스 확인

서비스에 남은 기록을 바탕으로 사용 현황과 문제를 살펴봅니다.

서비스 기록 조회 → 사용량·실행·표본 확인 → 문제 파악·검토

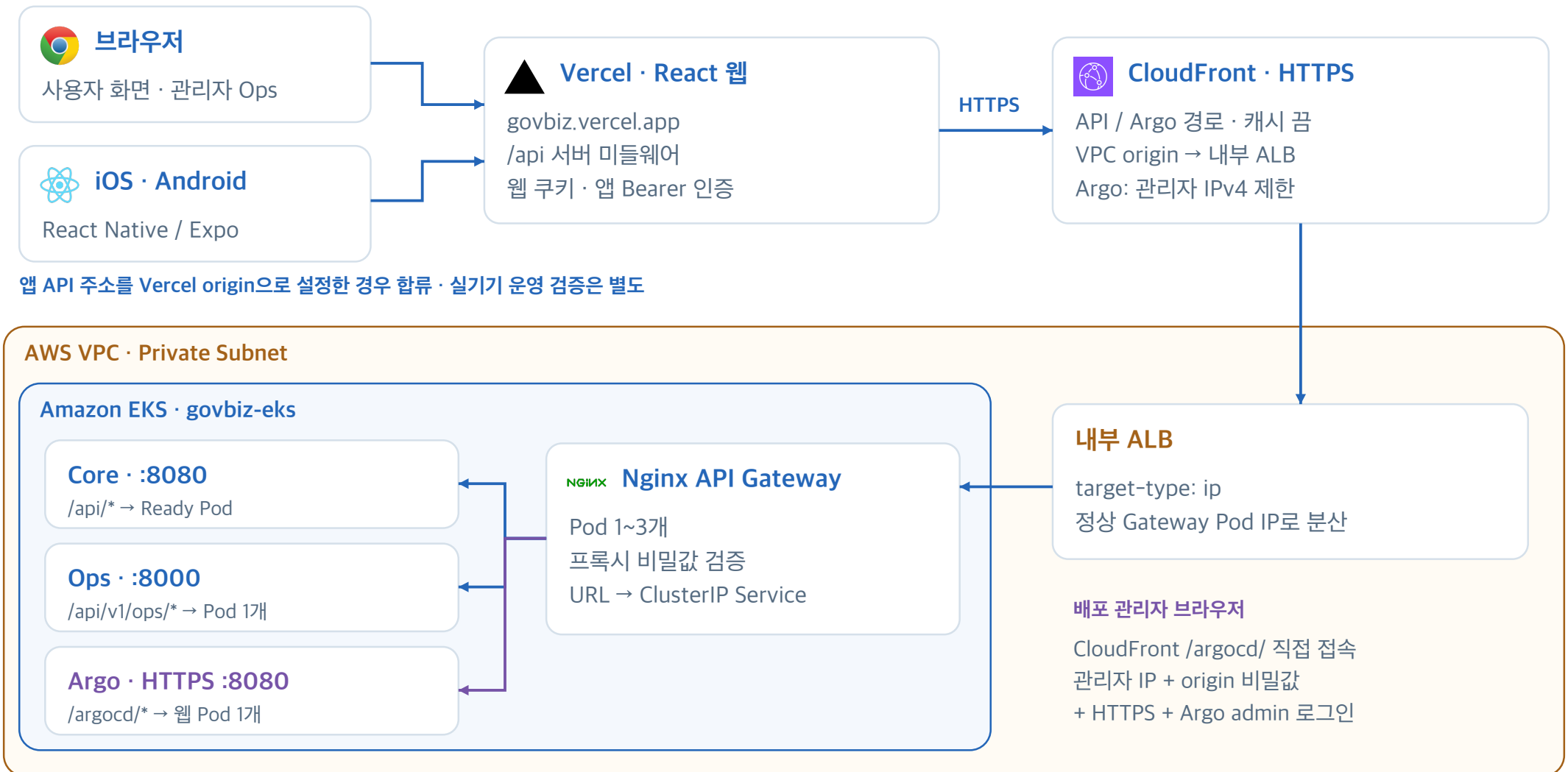
## 테스트 · 평가 결과 비교

준비한 평가 자료로 실행하고 결과를 검토해 변경 효과를 비교합니다.

평가 자료 준비 → 평가 실행 → 결과 검토·비교

# 웹·앱에서 백엔드까지 요청이 들어오는 경로

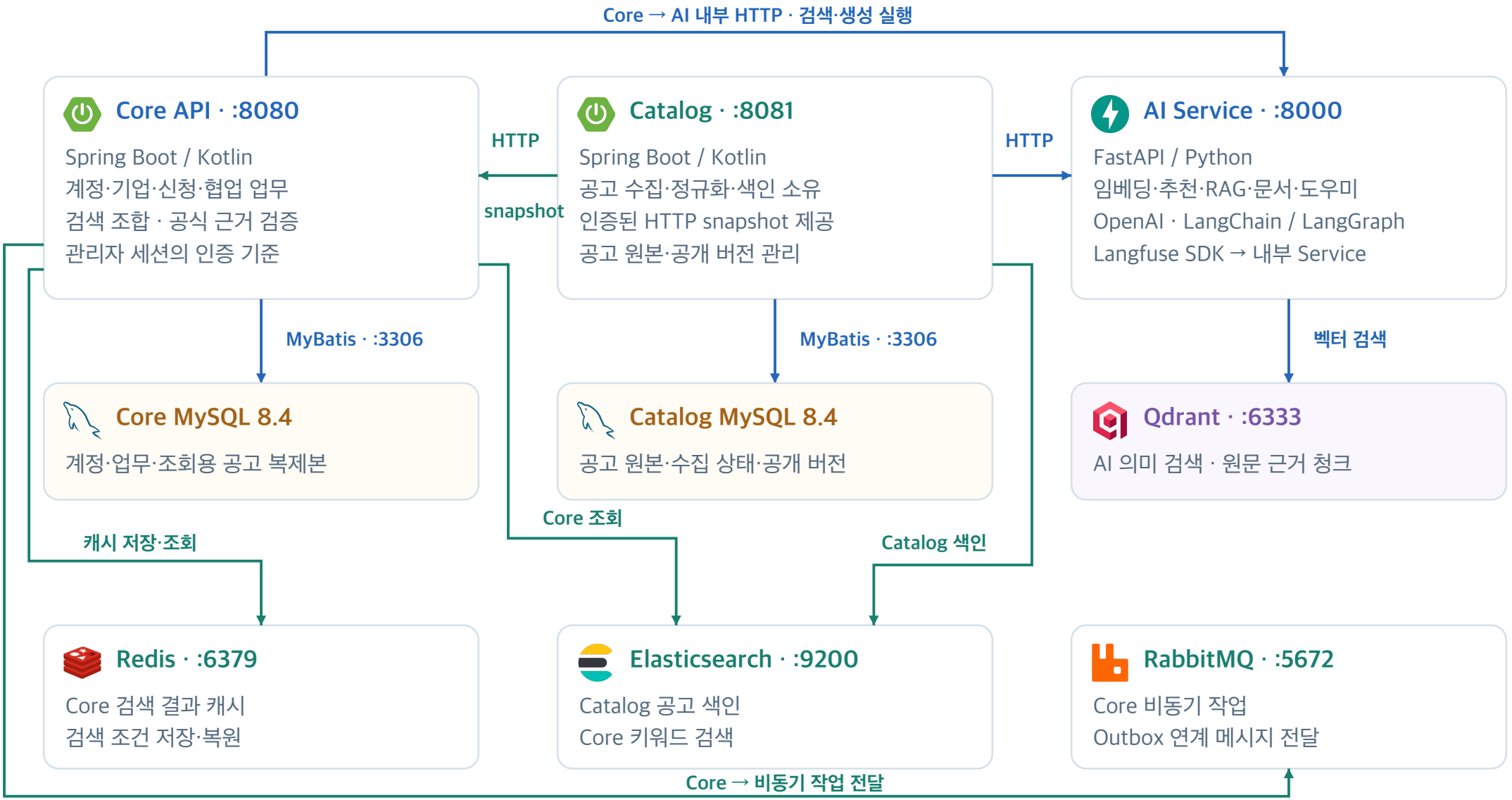
사용자 화면과 관리 화면의 진입점을 분리하고, Gateway에서 URL별 내부 서비스로 연결합니다.




업무 웹은 Vercel, Argo 웹은 EKS에서 제공 · Prefect / Langfuse UI는 기존 터널로 접근합니다.

# 업무 서비스가 호출하고 데이터를 저장하는 방식

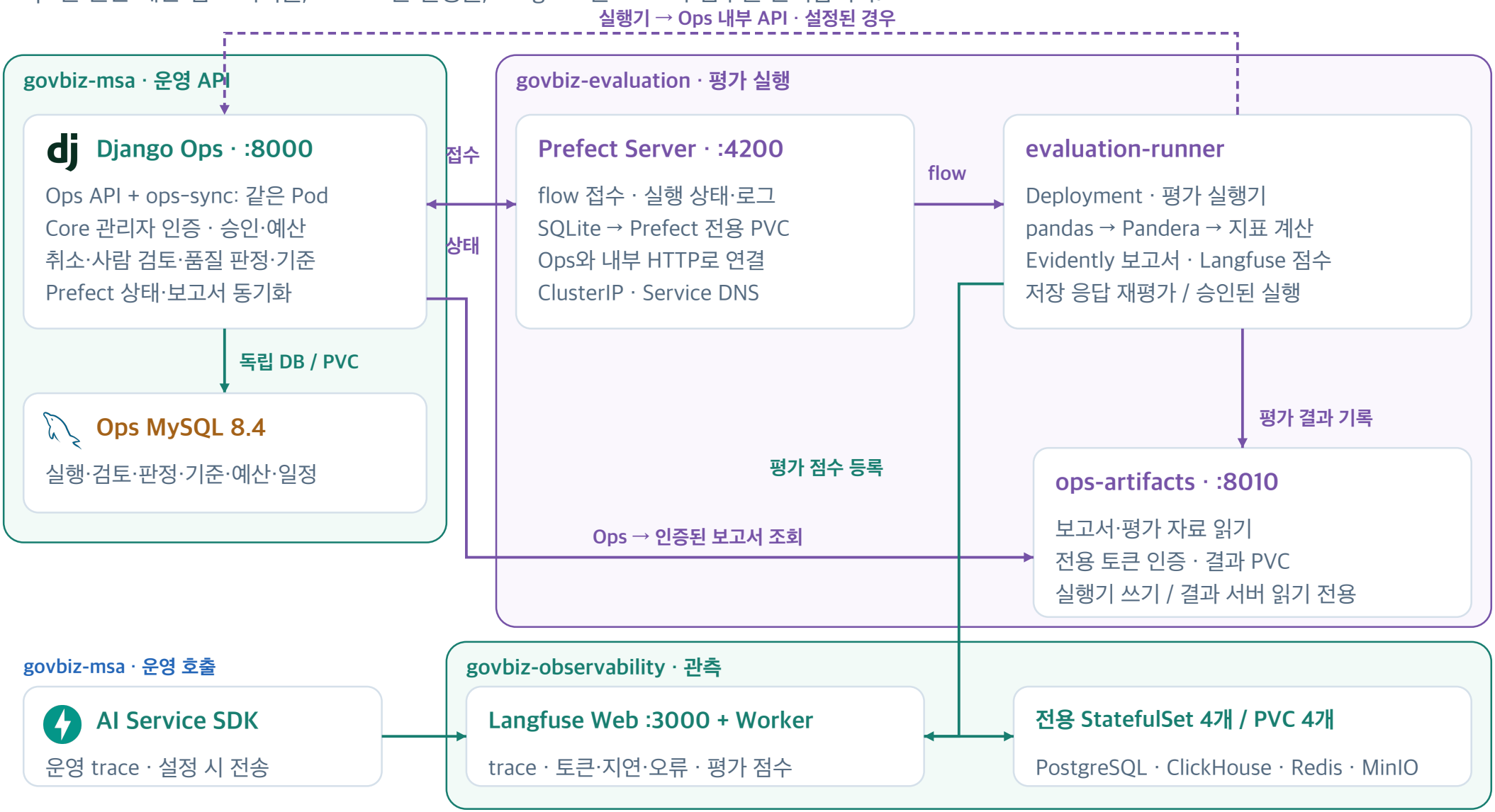
govbiz-msa의 Core·Catalog·AI를 중심으로, HTTP 호출과 데이터 저장·조회 경로를 구분합니다.




Core는 Catalog DB를 직접 읽지 않고 snapshot을 반영 · Ops와 독립 DB는 03, 외부 API 연결은 05에서 확인합니다.

# 평가 실행·결과 관리와 운영 추적의 연결

Ops는 권한·예산·검토 이력을, Prefect는 실행을, Langfuse는 trace와 점수를 관리합니다.




NetworkPolicy + Secret 인증 · 결과 PVC는 같은 노드에서 RWO 공유 · 관측 저장소는 업무 DB·Redis와 분리, Retain 적용.

# Pod를 어느 서버에 놓고 어떻게 늘리는가

EKS 워커는 같은 AZ의 EC2 기본 2대·최대 3대이며, Core·AI는 같은 서비스끼리 노드를 분산합니다.

**AWS Sydney / ap-southeast-2 · 워커 AZ: ap-southeast-2a**

### application · EC2 1대

업무·저장소 노드

- Core Pod 1
- AI Pod 1

Catalog · Ops(API + sync)
Core / Catalog / Ops MySQL
Redis · Elasticsearch · Qdrant
RabbitMQ

### observability · EC2 1대

관측·평가·Argo + 분산 Pod

- Core Pod 2
- AI Pod 2

Langfuse Web·Worker + 저장소
Argo CD Pod 5개
Prefect · 평가 실행기 · 결과 서버
실행기·결과 서버: 같은 노드

### application · 추가 EC2 1대

확장 조건이 충족되면 생성

- Core Pod 3
- AI Pod 3

Core / AI 세 번째 Pod 배치
같은 서비스는 EC2당 최대 1개
Gateway는 HPA로 1~3개
자원·배치 조건에 맞춰 스케줄

**확장 순서 · HPA와 Cluster Autoscaler는 서로 다른 대상을 조절**

**HPA → Pod 개수**
CPU requests 대비 평균 60%
Core·AI 2~3개 / Gateway 1~3개

→

**배치 대기 Pod**
기존 노드에 자원 부족
또는 분산 조건 불충족

→

**Cluster Autoscaler**
application 노드 1대 추가
준비 후 대기 Pod 실행

→

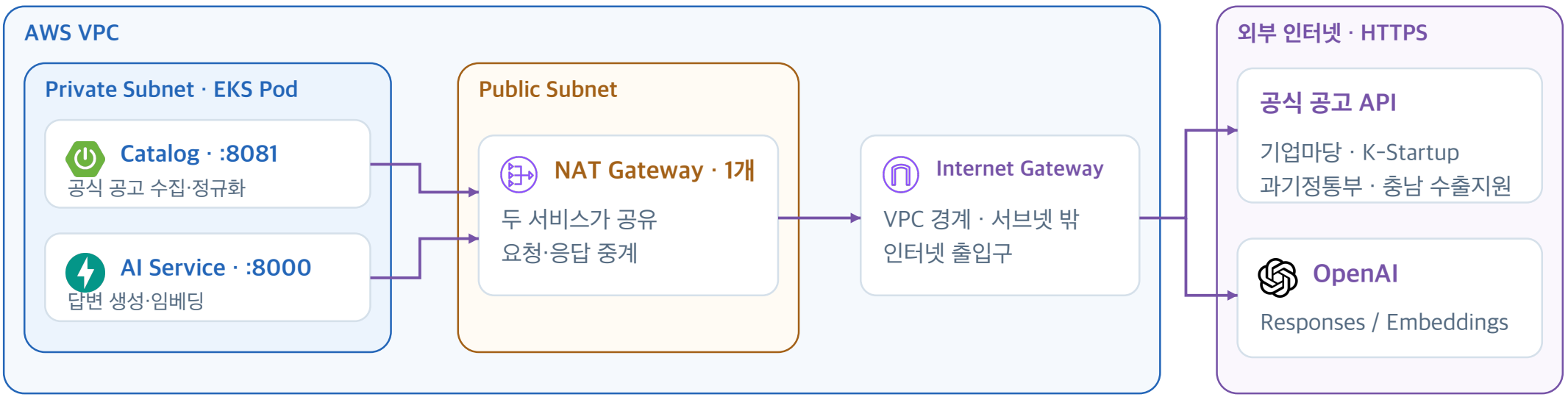
**총 EC2 2~3대**
application 1~2대
observability 1대

축소: HPA 5분 안정화 · 추가 노드 10분 불필요 + 안전한 이동 조건 · PDB로 자발적 중단 제한

Namespace는 논리 구분, EC2는 실행 위치 · EKS 제어면 서브넷은 2개 AZ, API는 private + 관리 IP 제한 public입니다.

# 외부 API 통신과 영속 데이터가 놓이는 곳

사설 Pod의 외부 요청은 NAT를 거치고, 업무·평가·관측 상태는 EBS gp3 PVC에 보존합니다.




Pod가 시작한 연결의 응답만 반환 · NAT로 외부에서 Pod에 새 연결을 시작할 수 없음

### 클러스터를 지원하는 AWS 저장소

**EBS gp3 · 데이터 보존**

EBS CSI → 암호화 PVC 13개
업무 7 · 평가 2 · 관측 4
StorageClass: Retain · AZ 종속
MySQL은 EKS StatefulSet 유지

**ECR · private 이미지**

CodeBuild가 이미지 5개 발행
EKS 노드가 digest로 pull
immutable git-SHA 태그
검증 소스와 실행 이미지 연결

**S3 · 이전 백업 보관**

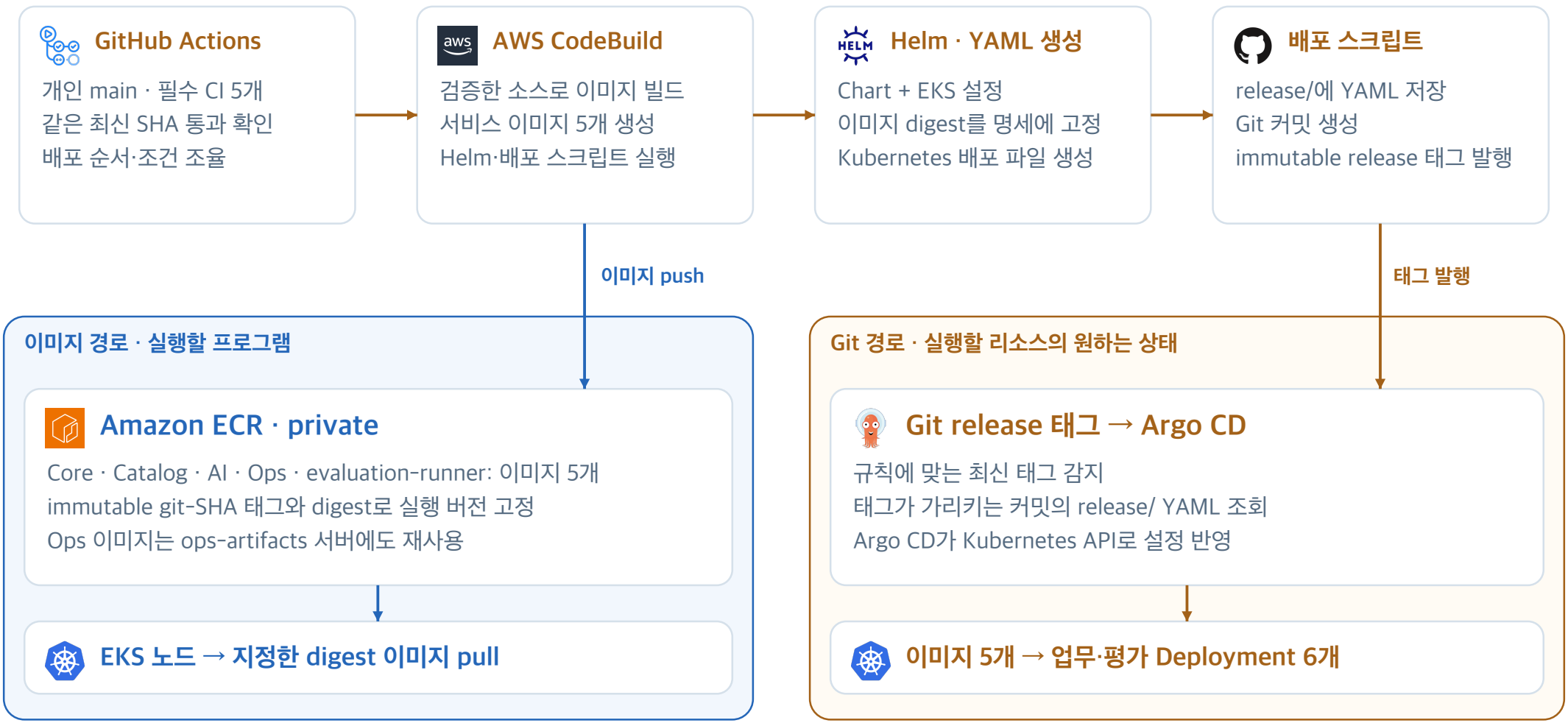
암호화 백업·메타데이터
비공개 · 버전 관리 · SHA-256
이전 백업 → EBS 데이터 복원
상시 DB 백업·자동 복제와 구분

API 키·제공처 활성화 설정 필요 · 평가 실행기의 외부 모델 호출은 원본 Chart에서 비활성

인프라: eksctl · CloudFormation · IAM / Pod Identity · EBS CSI · LBC(Helm 설치)로 Ingress → 내부 ALB 구성.

# 검증한 코드에서 이미지와 배포 파일을 만드는 과정

개인 main의 같은 최신 SHA를 검증한 뒤, 실행 이미지는 ECR에, 배포 YAML은 Git 태그에 발행합니다.




Git은 배포 YAML을 전달하고 ECR은 실행 이미지를 전달합니다. Argo 구성 요소와 배포 완료 판정은 07에서 이어집니다.

# Argo CD가 실제 배포를 반영하고 확인하는 구조

Argo CD는 observability 노드의 별도 namespace에서 동작하며, 사용자 API 요청 경로와 분리됩니다.

**argocd namespace · observability EC2 · Pod 5개**

### repo-server

Git 태그의 release/ YAML 조회
리소스 내용을 controller에 제공

### application-controller

Git 설정과 클러스터 상태 비교
Kubernetes API로 앱 동기화

### ApplicationSet controller

Application 생성·관리 구성 요소

### Redis · Argo 내부 캐시

업무·Langfuse Redis와 별도

### argocd-server · 상시 배포 관리 웹

Gateway → HTTPS ClusterIP :8080 → Deployment / ReplicaSet → Pod 1개

**EKS 관리형 제어면 → 애플리케이션**

### Kubernetes API

Argo 권한 확인 · 원하는 상태 반영
EC2에 배치된 업무 Pod가 아님

### govbiz-eks Application

동기화 대상 Deployment 6개
Core · Catalog · AI · Ops
평가 실행기 · 결과 서버
Service · 배포 hook 등 반영
PreSync prepare / resume Jobs

**배포 완료 확인 → 같은 소스의 웹 전환**

### 리비전 · hook · 실제 이미지 · Ready 상태 검증

기대 Git 리비전과 성공 hook, 실제 digest, 앱 준비 상태를 확인합니다.

### Vercel 웹 전환

EKS 성공 후 같은 SHA 배포

자동 교체 범위는 업무·평가 앱 · DB / PVC / 네트워크 / Langfuse는 별도 관리 · Argo 웹 접근 제어는 01을 참고하세요.

원본 재구성 · 2026.10.13 저장소 설정 기준 · 현재 클러스터 실측 아님

출처: README / AWS EKS 시스템 아키텍처

# 사용자 의도에 따라 업무를 연결하는 LLM 기반 라우팅

공고를 선택한 뒤 자연어로 필요한 작업을 요청하는 예시입니다.




### 공고 원문 질문

선택한 공고에서 답변 근거 확인

### 신청 준비

준비 화면에서 사용자가 작성

### 중복 지원 검토

비교할 공고와 정보를 확인 후 실행

### 파트너 조회

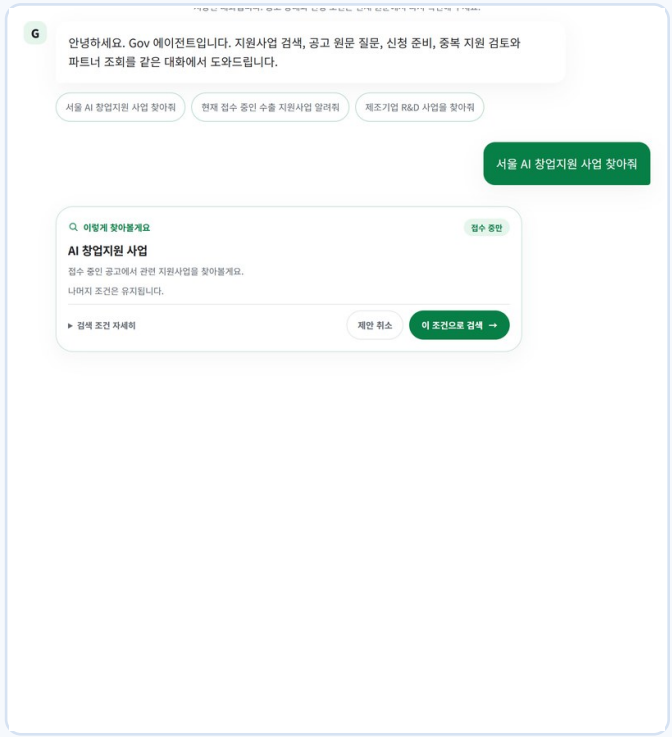
요청 조건에 맞는 모집글 확인

자연어 요청의 예시이며, 화면의 작업 버튼은 별도로 사용할 수 있습니다.

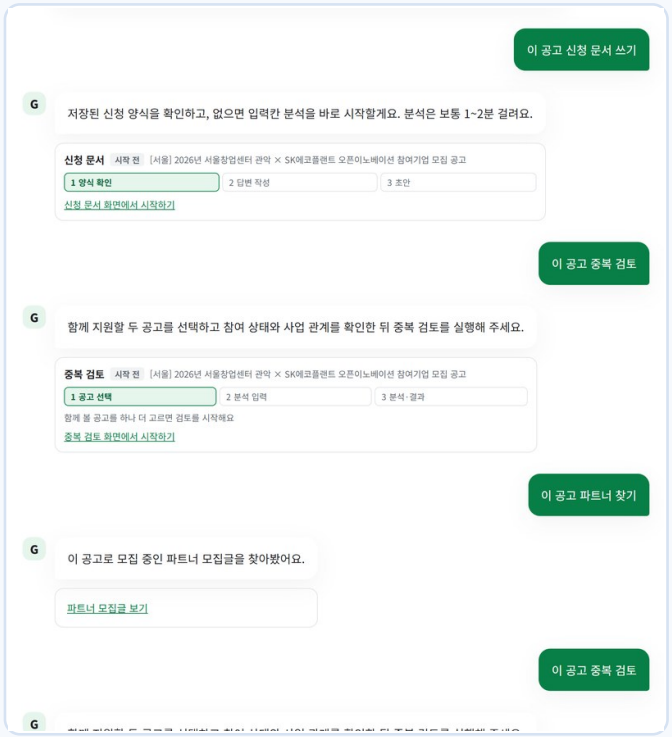
# 대화에서 확인하고, 작업 화면에서 실행한다

실제 관리자 화면: 조건 제안과 작업 카드를 통해 다음 행동을 명확하게 보여줍니다.

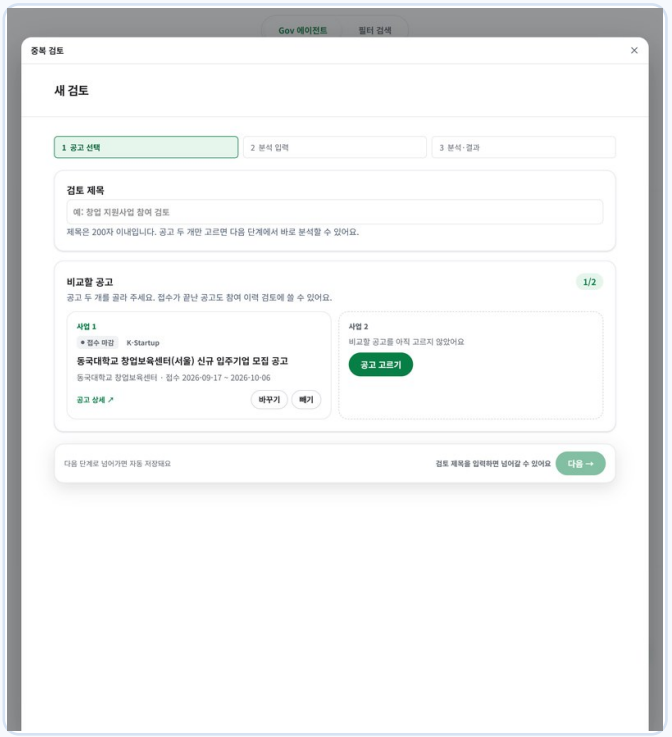
## 조건을 확인하고 검색




## 문서·중복 검토 카드




## 별도 검토 화면




## AI가 외부 제출·저장·삭제를 임의로 시작하지 않습니다.

카드에서 다음 화면으로 이동하고 사용자가 실행을 요청합니다. 작업 버튼으로 생성한 카드도 포함됩니다.

2026.10.15 로컬 웹의 저장된 대화 기록. 화면들은 기능 예시이며 하나의 연속 모델 실행을 의미하지 않습니다.

# 관리자 기능은 운영과 테스트로 나뉩니다

현재 React LLMOps 메뉴의 운영 3개·테스트 6개를 목적과 데이터 기준으로 구분했습니다.

## 운영 · 실제 서비스의 기록 확인

### 실사용 모니터링

AI 호출·토큰·비용 집계 / 같은 UTC 기간 비용 비교

### AI 실행 추적

기록된 실행의 단계·입출력·오류 확인

### 운영 표본 품질

저장 표본의 입력·근거·결과를 사람이 검토

## 테스트 · 평가 자료로 변경 효과 확인

### 성능 대시보드

평가 지표·변화 추이

### 평가 이력

결과·답변·보고서 검토

### 평가 자료 관리

원문·질문·채점 조건

### 새 평가

자료·모델 선택과 접수

### 예산 관리

평가 호출·토큰 한도

### 정기 실행

평가 일정 등록·중지

운영 화면은 수집된 호출·trace·표본을 보여줍니다. 성능 대시보드는 평가 자료의 결과이며, 서비스 전체 정확도를 뜻하지 않습니다.

## 테스트 실행 흐름

### 자료·예산 준비

원문·질문·실행 조건

→

### 평가 실행

수동·승인된 정기 실행

→

### 사람 검토·판정

근거·답변·품질 기준

→

### 비교 기준 지정

합격한 실행을 지정

# 운영 흐름 · 실제 서비스에서 남긴 기록을 확인

세 메뉴는 목적에 따라 각각 조회합니다. 사용량 기록과 개별 trace가 자동으로 연결되는 흐름은 아닙니다.




**운영 → 테스트로 이어지는 선택 경로**

상세 공고 답변 검토 요건 충족 → 회귀 후보 JSON 내보내기 → 별도 검토 후 평가 자료 편입

# 테스트 흐름 · 같은 조건으로 평가하고 비교

자료·모델·비교 대상을 고정하고, 실행 조건과 예산을 확인한 뒤 결과를 검토합니다.




실선: 진행 순서 · 점선: 선택·참조 경로. 성능 대시보드는 기준 지정 전에도 조회할 수 있습니다.

# 같은 조건에서 무엇을 다시 실행할 것인가

비교 목적에 따라 저장 응답, 고정 근거 답변, 고정 원문 RAG를 선택합니다.

REPLAY

## 저장 응답 재평가

저장한 답변·검색 결과

↓

지표·보고서를 다시 계산

**새 모델 호출 없음**

LIVE · ANSWER

## 고정 근거로 새 답변

같은 질문·같은 근거

↓

답변 모델을 새로 호출

**검색·임베딩 실행 없음**

LIVE · RAG

## 고정 원문으로 RAG

고정 원문·청크·평가 벡터

↓

질문 임베딩 → 검색 → 답변

**같은 청크 벡터는 재사용**

**비교 가능한 조건부터 고정합니다.**

자료 버전·모델·프롬프트·채점 조건을 기록하고, 새 호출은 승인된 전송 데이터와 호출·토큰 예산 안에서 실행합니다.

Replay는 현재 모델의 신규 품질 측정이 아닙니다. 고정 원문 RAG는 운영 수집·재청킹·검색 색인 전체의 품질을 뜻하지 않습니다.

# 실행 성공과 답변 품질을 구분한다

자동 지표를 계산한 다음에도 사람이 근거와 답변을 검토해야 비교 기준으로 사용할 수 있습니다.

01

## 자료·사례 검토

질문·원문·참조 조건
실제 답변·인용 대조

02

## 실행 검토 승인·판정

검토 근거와 품질 정책 고정
미검토 결과의 합격 차단

03

## 비교 기준 지정

합격한 실행만 관리자 지정
검토 변경 시 유효성 재확인

PRODUCTION

## 운영 관측

기록된 사용량·비용·trace 확인
성공 응답 중 수집한 표본을 사람 검토

EVALUATION

## 고정 자료의 성능 비교

같은 자료·조건의 모델 결과 비교
전체 실서비스 품질로 해석하지 않음

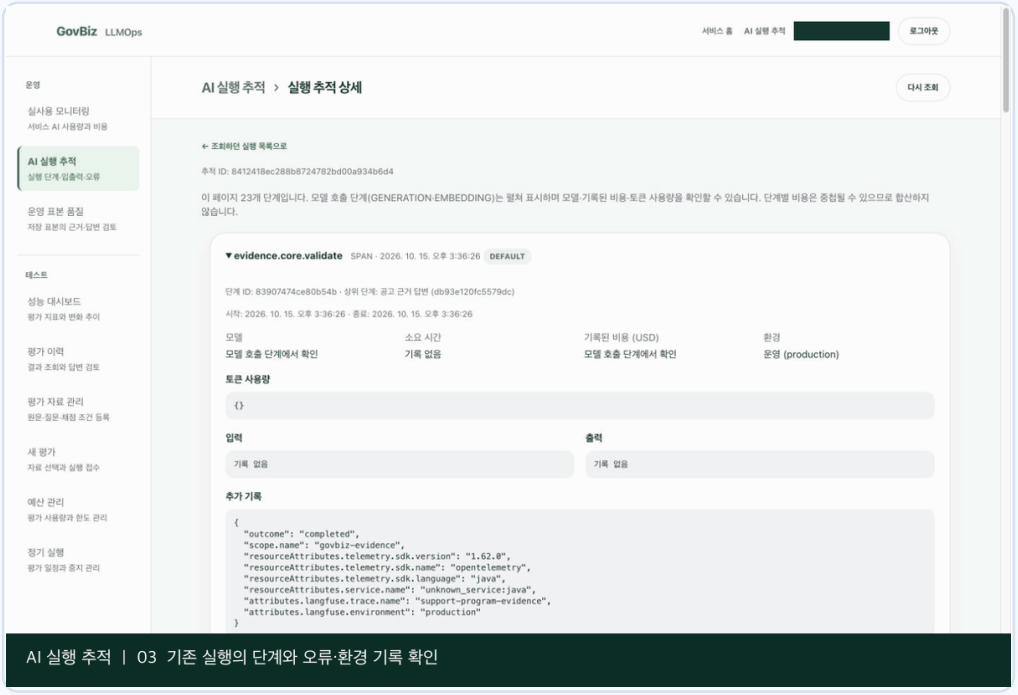
**취소·실패·사용량 미확정도 상태로 보존합니다.**

동시 요청은 예약량을 합산하고, 사용량 미확정을 0으로 정산하지 않습니다. 예산 단위는 호출 수·입출력 토큰입니다.

# 추적 화면과 평가 보고서를 함께 확인한다

React 운영 화면에서 실행의 단계·오류를 살피고, 별도의 평가 보고서에서 결과를 비교합니다.

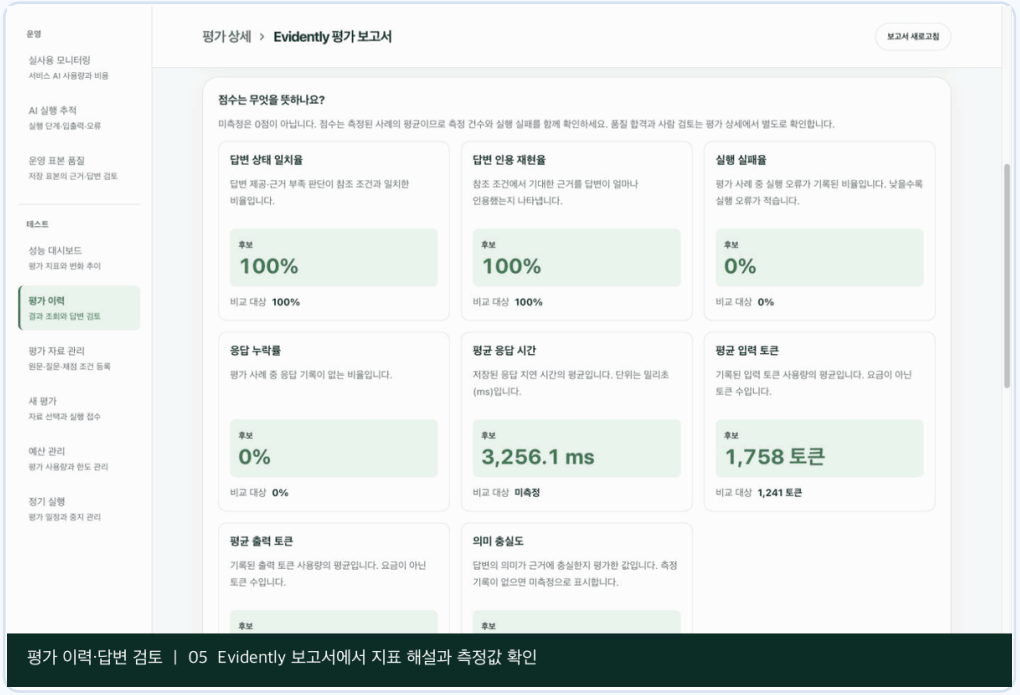
## AI 실행 추적




AI 실행 추적 | 03 기존 실행의 단계와 오류·환경 기록 확인

어느 단계에서 무엇이 실행됐는가?
모델·지연·토큰·오류 등 기록된 실행 정보를 확인합니다.

## Evidently 평가 보고서




평가 이력·답변 검토 | 05 Evidently 보고서에서 지표 해설과 측정값 확인

같은 평가 조건에서 어떤 결과가 나왔는가?
지표·분포를 확인하고 원문·답변 검토로 이어갑니다.

2026.10.15 운영 화면 기록. 화면의 수치는 당시 선택한 평가 결과이며, 현재 전체 서비스의 정확도를 뜻하지 않습니다.

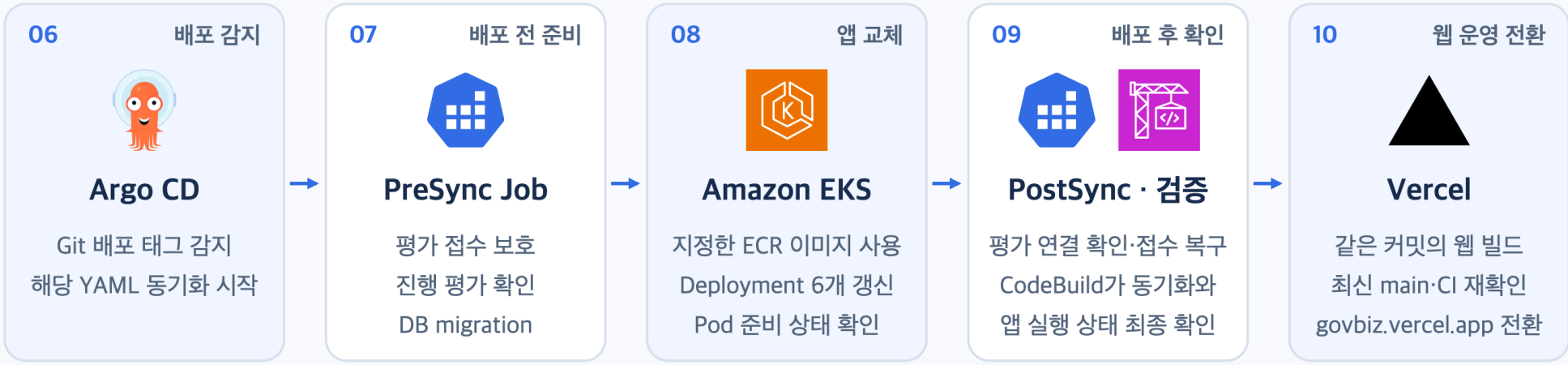
# EKS 반영을 확인한 뒤 웹을 전환한다

Argo CD가 Git 명세를 반영하고, 배포 리비전·hook·이미지·준비 상태를 확인합니다.

GOVBIZ / DEPLOYMENT / 02

## Git 배포 설정이 운영 서비스에 반영되는 과정

Argo CD가 백엔드를 동기화하고, 성공을 확인한 뒤 GitHub Actions가 Vercel 웹을 전환합니다.




백엔드 동기화·배포 Job·앱 준비 상태 확인이 실패하면 Vercel 운영 주소를 새 버전으로 전환하지 않습니다.

화살표: 배포 진행 순서 · HTTP 요청 경로와 구분

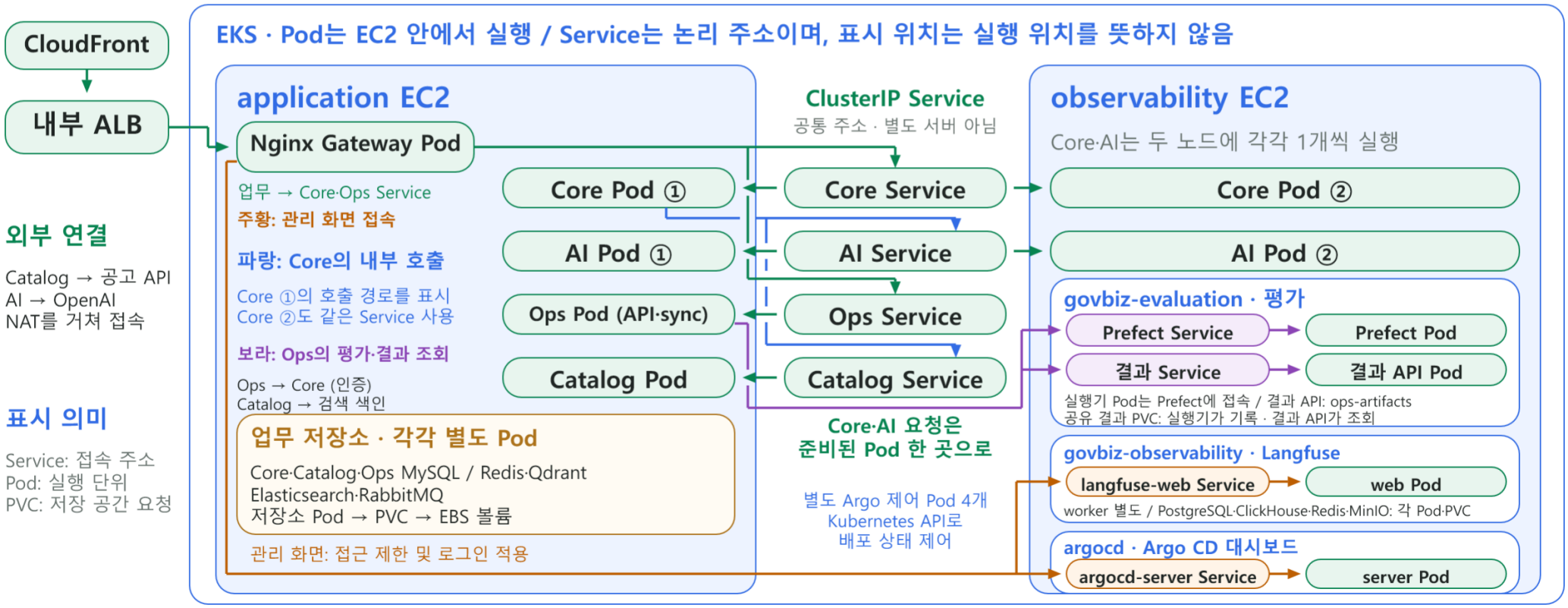
공식 브랜드 자산 · Kubernetes Contributors / CC BY 4.0

**검증한 소스와 실제 실행 버전을 연결합니다.**

CodeBuild는 클러스터 적용 대신 조회·검증을 수행하며, Argo CD가 Kubernetes 리소스를 동기화합니다.

# 서비스 주소와 실제 실행 위치를 구분한다

Service는 접속 주소이고, Pod는 EC2 노드 위에서 실행되는 단위입니다.




## Pod 분산

같은 서비스의 Pod를 두 노드에 분산

## 데이터 보존

저장소 Pod는 PVC·EBS에 연결

## 운영의 경계

평가·관측·배포 관리 역할 구분

# 수집이 실패해도 기존 공고를 지우지 않는다

전체 수집과 검색 준비가 끝난 공고 묶음만 Core의 조회용 데이터로 반영합니다.

01
### Catalog 수집
제공처 전체 페이지 수집
건수·완전성 검증

→

02
### 검색 준비·게시
Elasticsearch·벡터 준비
snapshot 리비전·지문
확정

→

03
### Core 계약 확인
HTTP로 전체 자료 수신
제공처·건수·버전 검증

→

04
### 짧은 DB 반영
조회용 데이터·checkpoint
한 transaction으로 반영

### 성공한 묶음만 반영
제공처 + 원본 ID로 식별하고 같은 입력을 다시 처리해도 결과를 유지합니다.

### 중간 실패는 기존 자료 보존
외부 API 오류·일부 페이지 누락·계약 불일치 시 기존 공개 공고를 유지합니다.

네트워크 수집은 DB transaction 밖에서 완료합니다. Core 복제본에는 동기화 지연이 있을 수 있습니다.

# 같은 업무 계약, 서로 다른 플랫폼 구현

React와 React Native가 업무 규칙·API 클라이언트·응답 검증을 공유합니다.

WEB

## React

브라우저 UI·상태
쿠키 세션
coreFetch
credentials: include

→

@GOVBIZ/SHARED

## 공통 업무 계층

도메인·유스케이스·오류
공통 API 클라이언트
Zod 응답 검증·DTO 변환
React 의존성 없음

←

MOBILE

## React Native

네이티브 UI·세션 저장
Bearer 인증
앱용 fetch
credentials: omit

**플랫폼이 API 주소·fetch·인증 전송 방식을 공통 클라이언트에 주입**

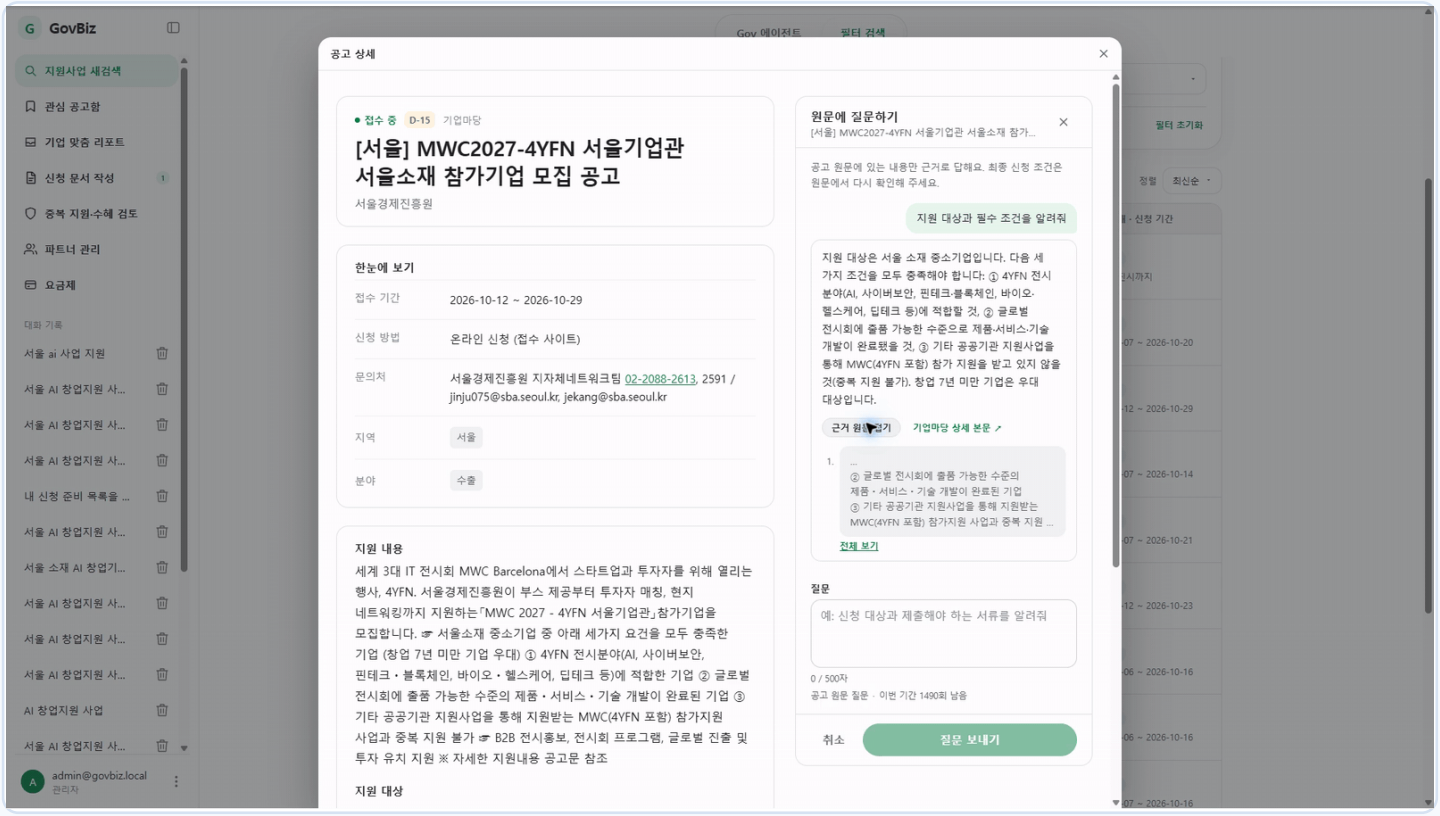
업무 규칙의 변경은 공유하고, 화면·내비게이션·저장 방식은 각 플랫폼에 유지합니다.

pnpm workspace · 하나의 lockfile | 일부 API는 플랫폼에 남아 있으며 모든 화면을 공유하는 구조는 아닙니다.

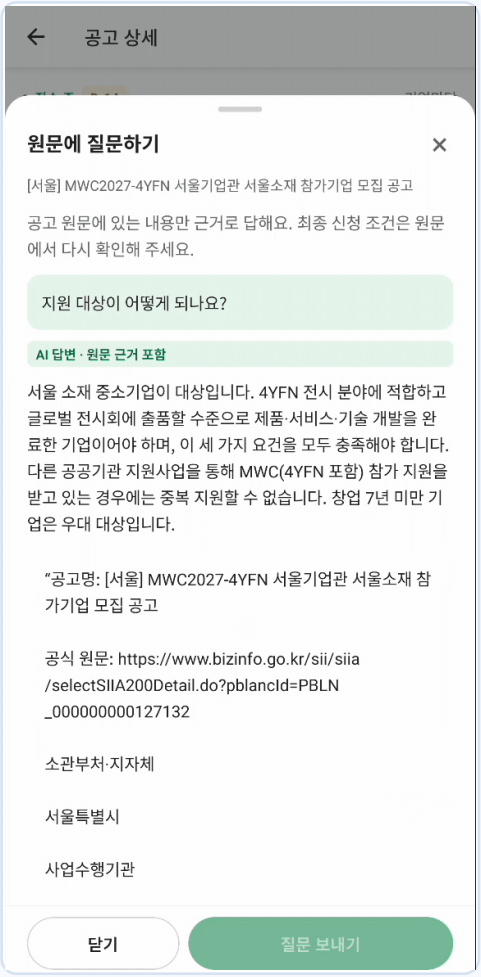
# 화면은 달라도 같은 응답 검증을 통과한다

웹·앱은 같은 근거 답변 계약을 검사합니다. GovAgent 응답에서는 선택 공고 ID도 대조합니다.

## React 웹




## React Native 앱




## Zod 응답 구조·답변 상태·공식 인용 URL을 공통 검증

웹 2026.10.14 · 앱 2026.10.15 화면 기록. 동일한 연속 요청이나 현재 성능 측정은 아닙니다.

# 부록 · README 전체 시스템 아키텍처

본문 7장은 원본의 구성 요소와 연결을 주제별로 재배치했습니다. 원본은 아래에서 전체 관계를 확인할 수 있습니다.

